

Request and Assignment Board Performance: Measured Client-Side Bottleneck, Sequence-Paginated Page Background Task Loading & Two Bugs Found In Testing

Business Central AL extension — DHTMLX-based control add-in
(page 50710 "DHX Request Assignment Board", menu caption "Requested/Assigned")

Scope	Planning-data JSON generation and client-side ingestion for the "Request" (Sequences) and "Assignment" (Resource timeline) panes
Objects touched	Codeunit 50604, Page 50710, Control Add-in interface + wrapper.js, Test Codeunit 60027
Objects added	Codeunit 50721 "ReqAssign BG Day Task Lines"
Platform feature used	Business Central Page Background Tasks — third application this session of the pattern first proven on the Gantt add-in's resource panel, then Task Scheduler
Related prior work	Gantt-Sync-to-Async-PageBackgroundTask-Enhancement.pdf and TaskScheduler-Performance-PageBackgroundTask-Enhancement.pdf — this document reuses their proven delivery mechanism without re-deriving it, and reports where this add-in's bottleneck genuinely differed from both
Prepared	2026-09-01

Contents

1. Executive Summary

2. Problem Statement — measured, not assumed: where the time actually goes

3. Solution Architecture — Part A: the N+1 fix

4. Solution Architecture — Part B: explicit scope cutoff + sequence-paginated loading

5. Implementation Detail — Codeunit 50604, Codeunit 50721, Page 50710

6. Implementation Detail — Control Add-in & wrapper.js

7. Two Bugs Found During Testing (and fixed before sign-off)

8. Verification — measured before/after

9. Object Inventory

10. Known Limitations & Follow-ups

11. Appendix A — Page Background Task API Reference

12. Appendix B — Glossary of Terms

13. Appendix C — How to Verify This Yourself

1. Executive Summary

Page 50710 "DHX Request Assignment Board" ("Requested/Assigned" in the UI) renders two linked DHTMLX Scheduler timelines — a "Request" pane (Sequences grouped from Day Task Lines) and an "Assignment" pane (a per-resource capacity timeline) — with live drag-and-drop between them, an undo stack, and accept/reject workflows. Users reported it as unacceptably slow to open. Rather than assume the cause (as would have been reasonable given the two prior optimizations this session both targeted server-side query patterns), the actual bottleneck was measured first, per explicit instruction.

The measurement overturned the initial assumption: a throwaway AL diagnostic timed the server-side JSON builder directly against the real dataset (7,004 Day Task Lines, a 30-workday window) at **3.2 seconds** for a 3.1 MB payload — already fast. The full page load was taking 25-35 seconds. The remaining ~22-32 seconds was **client-side**: a synchronous `JSON.parse()` of the whole payload, several full passes over all 7,004 lines to build the tree and DHTMLX event arrays, and a full `clearAll()` / `parse()` of DHTMLX's internal event store for each of the two panes — all before the first pixel could repaint.

The fix still uses the same Page Background Task mechanism proven twice already this session, but for a different reason than either prior case: here it exists to cut the amount of data the client has to synchronously ingest before first paint, not to move server query cost off the interactive session (that cost was already negligible). The first ~1,200 Day Task Lines (grouped by whole "sequence" — the atomic unit this board's drag-and-drop and accept/reject logic operates on, never split across the page boundary) render synchronously; the remainder is fetched by a Page Background Task and appended in place once ready.

Two real bugs were found and fixed during this work before it was considered done: a test-fixture date collision that made two new correctness tests fail against real production data (not a production defect), and a genuine client-side display bug where the "N of M Day Task Lines" counter stayed frozen at the first page's count after the background remainder had already arrived and merged correctly. Both are detailed in Section 7, with the second confirmed via direct in-page state inspection (not just visual inspection) before and after the fix.

~30s

Before — full synchronous load,
no interaction possible until done

~7s

After — time to first interactive
page (measured, two runs)

~13s

After — time to full 7,004-row
dataset (measured, two runs)

2. Problem Statement

2.1 The measurement, not the guess

Before writing any fix, a throwaway AL test codeunit (deleted immediately after use) called `ReqAssign_BuildPlanningDataJson` directly — the exact same 30-workday window `page 50710`'s own `GetDefaultWindow` computes — and timed it with `CurrentDateTime`, surfacing the result through a deliberately-failing test assertion (a standard AL diagnostic technique, since `Message()` output isn't otherwise visible from an automated test run):

```
TIMING: ReqAssign_BuildPlanningDataJson elapsed=3183ms jsonLen=3106347 chars StartDate=31-08-26
EndDate=09-10-26
```

3.2 seconds, 3.1 MB, 7,004 Day Task Lines. Separately, two live Playwright timing runs against the unmodified page measured **34.9s** and **24.9s** wall-clock from navigation to the toolbar showing "7004 of 7004 Day Task Lines". The gap between "3.2s to build the data" and "25-35s until the page is usable" is the actual problem, and it lives entirely on the client.

2.2 Where the client time actually goes

Reading `wrapper.js`'s `SetPlanningData` (the function AL calls with the full JSON payload) shows the full cost chain, all synchronous, all before first paint:

1. `JSON.parse()` on the ~3.1 MB string.
2. `.map()` over all 7,004 `dayTaskLines` to build the in-memory model.
3. `rebuildRequestTree()` — one more full pass, grouping every line by `sequenceKey` into the `project→task→sequence` tree.
4. `renderAll()` → `renderRequest()` + `renderResources()`, each of which calls `.filter()` / `.map()` over the *entire* `dayTaskLines` array again to build DHTMLX's event arrays, then `scheduler.clearAll()`; `scheduler.parse(events)`; — a full internal-store rebuild for up to ~7,000 events, *per pane*.

DHTMLX'S OWN DOM RENDERING IS NOT THE BOTTLENECK

Both scheduler instances are already configured with `smart_rendering: true` — DHTMLX's native row/column virtualization, confirmed present in `wrapper.js`'s `createRequestScheduler` / `createResourceScheduler`. DOM nodes are only created for what's in the viewport. The cost is entirely in the JS-side data processing and DHTMLX's internal (non-DOM) event-store ingestion that happens before any of that virtualized rendering can even start.

A second finding, out of scope for this pass but worth recording: `renderAll()` runs not only on initial load but after all six mutation triggers (accept/reject/assign/move/resize/unassign) — so every drag-drop or accept/reject action today re-pays a version of this same full-dataset cost. See Section 10.

3. Solution Architecture — Part A: the N+1 fix

`ReqAssign_BuildResourcesJson` had a genuine, if minor, N+1: one `Res.FindSet()`, then a fresh `ResSkill.SetRange("No.", Res."No."); ResSkill.FindSet()` per resource. Every other `ReqAssign_*` procedure already used `SetLoadFields` and/or dictionary caching — confirmed by reading them all; this was the one exception.

Fixed with a single merge-join pass instead of a Query object: both `Resource` and `Resource Skill` (filtered to `Type = Resource`) are scanned once, each in their own unmodified primary-key order, which guarantees a given resource's skill rows are contiguous in the skill scan — so the two cursors are walked in lockstep, one skill-pass total instead of one-per-resource. This was a deliberate choice over a Query object for this specific case: resource/skill row counts here are modest (dozens, not thousands), and a merge-join over two already-available sorted cursors is simpler than defining a new Query object for a join that doesn't need SQL-side aggregation. (Compare Task Scheduler's Part A, where a genuine multi-pass repeated-query algorithm justified an actual query-object-style rewrite — not every N+1 needs the same tool.)

4. Solution Architecture — Part B: Explicit Scope Cutoff + Sequence-Paginated Loading

4.1 The hazard found before it could ship: an implicit "I know everything" assumption

Research into `wrapper.js` before writing any pagination code found exactly one place that would silently break if fed an incomplete dataset: `defaultGlobalSelectionEndDate()` derives the "30-workday scope cutoff" — used to gate what counts as "in scope" for accept/reject, and to decide whether a sequence-drag should be blocked — by scanning every `dayTaskLines` entry for distinct dates. A naive "first N rows" page could omit some of the 30 workdays and silently compute the wrong cutoff, corrupting scope logic before the background remainder even arrived.

The fix removes the hazard at its root rather than working around it: AL already computes this exact date (`page 50710` 's `GetDefaultWindow` 's `EndDate`). It is now sent explicitly as a new `scopeEndDate` field on the JSON root, and `wrapper.js` 's `SetPlanningData` uses it directly instead of re-deriving it from a dataset that may not yet be complete:

`wrapper.js`, `SetPlanningData` (one-line change)

```
globalSelectionEndDate = data.scopeEndDate ? parseDateOnly(data.scopeEndDate) : null;
```

This makes every later pagination decision safe by construction — the client never again needs "complete" data to get this value right, regardless of how the payload is ever chunked in the future.

4.2 Pagination unit: whole sequences, never split

Mirroring the "never split the atomic grouping unit" rule already established for Task Scheduler (there: `Job`; here: `sequenceKey`, since that is the unit `rebuildRequestTree()` groups by and the unit accept/reject/drag operate on) — the new `ReqAssign_BuildPlanningDataJson_Paged` groups Day Task Lines by `sequenceKey` in the same order the existing code already establishes, and accumulates whole sequences into the first page until `MaxLines` is reached. The sequence that crosses the line is included in full — never split mid-sequence. A cheap pre-scan (key fields only) decides page membership before any full line JSON is built, so excluded rows never pay for the Job/Job Task description lookups either.

Page size chosen: 1,200 — roughly 17% of the measured 7,004-row dataset. Large enough that the visible Sequences tree and near-term timeline are immediately populated and interactive; small enough to cut the client-side parse/model-build/DHTMLX-ingest cost meaningfully for first paint. See Section 8 for how this held up against measurement.

4.3 Delivery — the same proven mechanism, unchanged

- 1 **Synchronous first page.** `RefreshPlanningData` (the one shared entry point already used by `ControlReady` and `OnRequestReset`) calls the paged builder and renders the first page via the existing,

unmodified `SetPlanningData` .

- 2 **Enqueue.** If any sequences remain, `EnqueueDayTaskLinesBackgroundTask` calls `CurrPage.EnqueueBackgroundTask(...)` against the new Codeunit 50721 and returns immediately.
- 3 **Background child session.** Codeunit 50721 "ReqAssign BG Day Task Lines" — structurally copied from Codeunit 50713 (Gantt) / Codeunit 50720 (Task Scheduler) — reads its parameters, calls the new `ReqAssign_BuildDayTaskLinesJson_ForKeys` for just the remaining sequences, and returns via `Page.SetBackgroundTaskResult()` .
- 4 **Completion callback (stash only).** `OnPageBackgroundTaskCompleted` stashes into a plain page variable — confirmed, not re-verified, that control add-in calls are rejected from this trigger (already proven twice this session on the other two add-ins).
- 5 **Client poll → append.** `wrapper.js`'s bounded 500 ms/60-attempt poll (same shape as the other two add-ins) calls `OnPollDayTaskLinesResult` , which delivers the pending JSON into the control add-in via the new `AppendDayTaskLines(dayTaskLinesJson)` — which `.push()` es the incremental batch into the existing `dayTaskLines` array and calls `rebuildRequestTree()` + `renderAll()` once for the merged set. It never routes through `SetPlanningData` itself — that would re-run the full-reset path (clearing selection/undo state) this feature exists to avoid paying twice.

This page has no Previous/Next/date-filter navigation (`RefreshPlanningData` always rebuilds the same current-week-plus-30-workday window), so the staleness guard needed here is simpler than Task Scheduler's: a `TaskId` comparison alone is sufficient — there is no separate "did the displayed period move on" re-check to duplicate.

5. Implementation Detail — Codeunit 50604, Codeunit 50721, Page 50710

5.1 One shared line-JSON builder, three callers

The per-line JSON construction (previously duplicated logic risk) was extracted into a single shared local procedure, `ReqAssign_BuildDayTaskLineObj`, used by all three variants: the original (unpaged) `ReqAssign_BuildDayTaskLinesJson`, the new `ReqAssign_BuildDayTaskLinesJson_Paged`, and the new `ReqAssign_BuildDayTaskLinesJson_ForKeys` (public, background-task-callable, and independently unit-testable). One source of truth for line-JSON shape across all three call paths — a safe, behavior-preserving refactor, verified by the new set-equality test in Section 8.

5.2 Codeunit 50721 "ReqAssign BG Day Task Lines"

Structured identically to its two predecessors this session (Codeunit 50713 for Gantt, Codeunit 50720 for Task Scheduler) — same `Page.GetBackgroundParameters()` / `Page.SetBackgroundTaskResult()` shape, same read-only child-session contract.

5.3 Page 50710

All additions live after the existing `OnRequestReset` trigger and after `RefreshPlanningData`'s existing body — confirmed via diff inspection that none of the six mutation triggers (`OnAcceptSequence`, `OnRejectSequence`, `OnAssignDayTaskLine`, `OnMoveAssignment`, `OnResizeAssignment`, `OnUnassignDayTaskLine`) or their corresponding codeunit 50604 procedures were touched by this change.

6. Implementation Detail — Control Add-in & wrapper.js

Purely additive to `DHXRequestAssignmentAddin.ControlAddin.al`: one new event (`OnPollDayTaskLinesResult`) and three new procedures (`AppendDayTaskLines`, `NotifyDayTaskLinesTaskPending`, `StopDayTaskLinesPolling`).

On the JS side, all changes sit in the last ~200 lines of a 6,724-line file — nowhere near the drag-and-drop optimistic-update state machine, undo stack, or capacity-overlay code (which live in the untouched first ~6,550 lines). The poll timer reuses the exact 500 ms/60-attempt shape already implemented twice this session, deliberately not re-tuned.

7. Two Bugs Found During Testing (and Fixed Before Sign-off)

7.1 Test-fixture date collision (test bug, not a production defect)

FIRST TEST RUN: 2 OF 3 NEW TESTS FAILED

`GivenSeqCrossesMaxLinesBoundary...` reported Sequence B with 0 lines instead of 3;
`GivenPagedFirstPagePlusBackgroundRemainder...` 's own sanity check expected 7 fixture lines back from the non-paged builder and got 17.

Root cause: both new tests declared `PlanDate := WorkDate()` — Business Central's "today" — for their date range. `ReqAssign_BuildPlanningDataJson` deliberately has no Job No. filter ("any Job/Task/Skill, no pre-filtering" — a pre-existing, correct design choice, confirmed via its own doc comment predating this session), so a `WorkDate()`-based query window landed squarely inside the real CRONUS NL sandbox's populated Day Planning date range, and the tests' unfiltered id-count assertions picked up real production rows alongside the 7 fixture rows.

Fix: a new local helper, `PagingFixtureIsolatedDate()`, returning a fixed far-future date (2050-01-01) — used consistently by `InsertPagingFixture` and both new test methods. The two pre-existing tests in the same codeunit were left untouched (they scope their own assertions by explicit Day Line No./id lookups rather than unfiltered counts, so they were never actually affected by the collision — confirmed by their passing both before and after this fix).

RESULT AFTER THE FIX

`al_run_tests` on codeunit 60027: **5 passed, 0 failed** — including proof that a sequence crossing the `MaxLines` boundary is included whole, and that first-page + background-remainder is set-equal (by id) to the non-paged builder's full output.

7.2 Stale "N of M Day Task Lines" counter (real client-side bug)

After the background remainder was confirmed to have arrived and merged correctly — verified by directly evaluating `dayTaskLines.length` in the page's own JS context via Playwright, returning **7004** — the toolbar's "N of M Day Task Lines in horizon" label was still showing the stale first-page count ("1203 of 1203"), unchanged for over 40 seconds after the real data had completed loading.

Root cause: the label is set by `updateSequenceScopeEditor()`, which reads `dayTaskLines.length` — but this function is only called from `SetPlanningData` 's initial-load path, not from the new `AppendDayTaskLines`. The underlying data was correct throughout; only its on-screen summary count was stale.

`wrapper.js`, `AppendDayTaskLines` — one line added at the end

```
rebuildRequestTree();
renderAll();
updateSequenceScopeEditor(); // ← added: refreshes the toolbar count + scope-editor date bounds
```

A no-argument call was used deliberately — it preserves whatever `activeScopeSequenceKey` /selection the user currently has, unlike `SetPlanningData`'s own `updateSequenceScopeEditor(null)` call, which is only correct on a full fresh load.

RESULT AFTER THE FIX

Re-tested live: the counter now correctly transitions from "1203 of 1203" to "7004 of 7004" once the background append completes, matching the underlying data at every point in between.

8. Verification — Measured Before/After

8.1 Static verification

Check	Result
<code>al_compile / al_build</code>	Succeeded, 0 errors (both before and after the two bug fixes)
Codeunit 60027 "Req Assign Seq Grouping Tests" (5 tests total, 3 new)	5 passed, 0 failed, 0 skipped
<code>git diff --stat</code> on <code>src/dhx/ganttdemo2/</code> and <code>src/dhx/projectschedule/</code>	Empty — confirmed untouched
Six mutation triggers + their codeunit 50604 procedures + wrapper.js drag/drop/undo/capacity-overlay code	Confirmed untouched by diff inspection — all additions land after existing code, no existing line altered

8.2 Live timing — measured, not projected

Two independent live Playwright measurements were taken against the unmodified (baseline) page, and two more against the fixed page, all against the real CRONUS NL dataset (7,004 Day Task Lines). The post-fix measurements used direct in-page state polling (checking `dayTaskLines.length` at 100 ms intervals from the moment navigation started) rather than screen-scraping the toolbar label, to eliminate tool-round-trip noise from the numbers.

Run	Metric	Result
Before, run 1	Full synchronous load (7004/7004 visible)	34.9s
Before, run 2	Full synchronous load (7004/7004 visible)	24.9s
After, run 1	Time to first interactive page (1203 rows in memory)	7.58s
After, run 1	Time to full dataset (7004 rows in memory)	14.98s
After, run 2	Time to first interactive page (1203 rows in memory)	6.57s
After, run 2	Time to full dataset (7004 rows in memory)	11.73s

HONEST INTERPRETATION

Time-to-first-interactive improved roughly **4x** (average ~30s before → ~7.1s after) — the number that actually matters for perceived responsiveness, since the page is usable (first sequences visible, scrollable, resources

shown) at that point, not at full-dataset completion. Time-to-full-dataset also improved, roughly **2.2x** (~30s before → ~13.4s after), likely because the first page's cheaper AL call and lighter client-side ingestion free up both the interactive session and the browser's main thread sooner, even though the background task is doing comparable total work to what the old synchronous call did. The improvement is real but more modest than the ~5-7x figure estimated in the implementation plan before measurement - a meaningful fraction of total load time is fixed platform/shell/DHTMLX-initialization overhead that doesn't shrink just because the payload does (see Section 10).

8.3 Functional/regression verification

Live-rendered content (Sequences tree, resource timeline, task bars) confirmed visually identical to the pre-change baseline screenshot for the same visible window. Browser console showed no new errors after either the initial fix or the two bug fixes - only pre-existing, unrelated platform noise (an avatar-photo 404, other-company OData 401s from the BC shell's company switcher) present before this work began too. The six mutation triggers and their AL/JS implementations were not exercised via live drag-and-drop against the shared CRONUS NL sandbox during this pass (to avoid mutating shared demo data without a clear reset path) - their safety rests on the stronger guarantee that none of their code was touched at all (Section 8.1), not on a live functional click-through.

9. Object Inventory

Object	Type	Status	Role
Codeunit 50604 "DHX Data Handler"	Codeunit	MODIFIED	<code>ReqAssign_BuildResourcesJson</code> N+1 fixed (merge-join); <code>scopeEndDate</code> added to the JSON root; new shared <code>ReqAssign_BuildDayTaskLineObj</code> ; new <code>ReqAssign_BuildPlanningDataJson_Paged</code> , <code>ReqAssign_BuildDayTaskLinesJson_Paged</code> , <code>ReqAssign_BuildDayTaskLinesJson_ForKeys</code> . The six mutation procedures untouched.
Page 50710 "DHX Request Assignment Board"	Page	MODIFIED	<code>RefreshPlanningData</code> now calls the paged builder (page size 1,200) and enqueues the remainder; new <code>EnqueueDayTaskLinesBackgroundTask</code> , <code>OnPageBackgroundTaskCompleted / Error</code> (stash-only, TaskId-gated), <code>OnPollDayTaskLinesResult</code> . Six mutation triggers untouched.
DHXRequestAssignmentAddin (Control Add-in)	Control Add-in	MODIFIED	+1 event (<code>OnPollDayTaskLinesResult</code>), +3 procedures (<code>AppendDayTaskLines</code> , <code>NotifyDayTaskLinesTaskPending</code> , <code>StopDayTaskLinesPolling</code>)
wrapper.js (request_assignment)	Add-in script	MODIFIED	New poll timer + <code>AppendDayTaskLines</code> (append-only, never routes through <code>SetPlanningData</code> 's full reset); <code>scopeEndDate</code> consumption in <code>SetPlanningData</code> ; <code>updateSequenceScopeEditor()</code> call added to <code>AppendDayTaskLines</code> (Section 7.2 fix). All changes in the file's last ~200 lines; drag/drop/undo/capacity-overlay code (first ~6,550 lines) untouched.
Codeunit 50721 "ReqAssign BG Day Task Lines"	Codeunit	NEW	Page Background Task target — builds the remaining sequenceKeys' Day Task Lines off the interactive request path
Test Codeunit 60027 "Req Assign Seq Grouping Tests"	Codeunit (Test)	MODIFIED	2 new tests + 1 new shared fixture helper added (5 total, all passing); 2 pre-existing tests untouched

Page 50620 "Gantt Demo DHX
2" / Codeunit 50613 / Page
50621 "DHX Scheduler
(Project)" / Codeunit 50604's
Task-Scheduler-only
procedures

—

unchanged

Explicitly out of scope for this pass - confirmed
untouched (Section 8.1)

10. Known Limitations & Follow-ups

- **The per-mutation `renderAll()` cost is unaddressed.** All six mutation triggers (accept/reject/assign/move/resize/unassign) still trigger a full `clearAll()` + `parse()` re-derivation over the complete, currently-loaded `dayTaskLines` set after every single interaction - not just on initial load. Once the background remainder has fully arrived, every subsequent drag-drop or accept/reject action pays a version of the same cost this document fixes for first paint. Rearchitecting DHTMLX event updates to be incremental (patch, not replace) instead of full `clearAll()`+`parse()` is a substantially larger, riskier change than anything done in this pass, and was explicitly out of scope per the plan.
- **A meaningful fraction of total load time is fixed overhead** (BC shell/iframe boot, DHTMLX scheduler initialization) that pagination cannot reduce further - this is why the measured improvement (~4x to first interactive, ~2.2x to full) is real but more modest than a naive "1200/7004 rows $\approx$ 6x less data" estimate would suggest.
- **Live drag-and-drop/accept/reject was not exercised against the shared CRONUS NL sandbox** during this verification pass, to avoid leaving shared demo data in a mutated state without a clean reset path. Confidence instead rests on the six mutation triggers and their entire code path (AL and JS) being provably untouched by this change (Section 8.1) - a structural guarantee, not a behavioral one. A follow-up pass in an isolated/disposable environment could add live coverage of all six triggers plus a drag-drop of a background-appended (not first-page) sequence specifically.
- **No dedicated timing regression test** exists for either the 3.2s AL-side baseline or the client-side numbers in Section 8.2 - both were captured manually for this document. A repeatable performance-regression harness (even a simple "assert elapsed < N ms" test, accepting some environmental flakiness) would catch a future regression in either layer automatically.

Appendix A — Page Background Task API Reference

Identical platform mechanism documented in both prior companion documents' Appendix A; reproduced here for a reader of this document alone.

Element	Signature / Note
<code>CurrPage.EnqueueBackgroundTask</code>	(var TaskId: Integer; CodeunitId: Integer; [Parameters: Dictionary of [Text, Text]], [Timeout: Integer], [ErrorLevel: PageBackgroundTaskErrorLevel])
<code>Page.GetBackgroundParameters()</code>	Called inside the background codeunit's <code>OnRun()</code> to read the input dictionary
<code>Page.SetBackgroundTaskResult(Result)</code>	Called inside <code>OnRun()</code> to return the output dictionary
<code>OnPageBackgroundTaskCompleted(TaskId; Results)</code>	Page trigger. Cannot call the control add-in directly - stash into plain AL vars only
<code>OnPageBackgroundTaskError(TaskId; ErrorCode; ErrorText; ErrorCallStack; var IsHandled)</code>	Page trigger, fired on failure/timeout of the background task
Session characteristics	Read-only (a write raises a permission error); own <code>OpenCompany</code> /transaction; canceled if the page closes; default 5 concurrent tasks per parent session; max 600,000 ms timeout

Appendix B — Glossary of Terms

Term	Meaning in this document
Sequence / <code>sequenceKey</code>	The atomic grouping unit of the "Request" pane - a thread of Day Task Lines sharing Job No./Job Task No./Skill/Sequence No. This board's drag-drop, accept/reject, and undo logic all operate at this granularity, which is why pagination never splits one across the sync/background boundary.
Time to first interactive	This document's primary UX metric (Section 8.2) - the moment the first page's data is fully in memory client-side and the board is usable, as opposed to the moment the entire dataset (first page + background remainder) has arrived.
Stash-and-poll	The delivery pattern required because a Page Background Task's completion trigger cannot call the control add-in directly: the result is stashed into plain page variables, then delivered later from a normal, JS-pollled trigger. Reused unchanged from the Gantt/Task Scheduler precedent.
<code>scopeEndDate</code>	New JSON field (Section 4.1) carrying the server-computed 30-workday scope cutoff explicitly, removing wrapper.js's prior need to derive it by scanning the complete <code>dayTaskLines</code> array - the fix that makes pagination safe regardless of how the payload is chunked.
<code>PagingFixtureIsolatedDate()</code>	Test-only helper (Section 7.1) returning a fixed far-future date, used to keep the new pagination tests' unfiltered assertions from colliding with real production Day Planning data in the shared test environment.

Appendix C — How to Verify This Yourself

1. Open `src/codeunit/codeunit_50604_DHXDataHandler.al` and search for `scopeEndDate` - confirm it's added to the root JSON object in both `ReqAssign_BuildPlanningDataJson` and `..._Paged` (Section 4.1).
2. In the same file, search for `ReqAssign_BuildDayTaskLinesJson_Paged` - confirm the pre-scan/inclusion logic never splits a `sequenceKey` group (Section 4.2).
3. Open `src/dhx/request_assignment/page_50710_DHXRequestAssignmentBoard.al` and search for `DayTaskLinesPageSize` - confirm the chosen value (1,200) and that `RefreshPlanningData` is the only caller of the paged builder.
4. Open `src/dhx/request_assignment/wrapper.js` and search for `AppendDayTaskLines` - confirm it ends with `updateSequenceScopeEditor()` (Section 7.2's fix) and never calls `SetPlanningData`.
5. Run `test/RequestAssignmentSequenceGrouping.Test.Codeunit.al` (codeunit 60027) via `al_run_tests` - confirm 5 passed, 0 failed.
6. Open the live Request and Assignment Board and, in the browser devtools console (or via an automated tool), evaluate `dayTaskLines.length` shortly after open, then again ~15s later - confirm it grows from ~1,200 to the full dataset size, and that the toolbar's "N of M Day Task Lines" label tracks it at both points.